MIZAN.AI — INTERNAL ENGINEERING REFERENCE

Offline Ingestion, GCP Deployment & Public Domain Setup

A step-by-step walkthrough of how the Know VAT & ZATCA knowledge base was built,
how the full stack was deployed to Google Cloud, and how it was made public at mizanai.org.

Prepared for: Mohammed (Mizan.ai)

Scope: Offline RAG ingestion pipeline · Kubernetes deployment on GCP (k3s) · Cloudflare domain, DNS & TLS

Purpose: Reference for personal understanding, technical interviews, and live demos

Contents

PART 1

Offline Ingestion Pipeline

Mizan.ai's "Know VAT & ZATCA" feature answers questions about Saudi VAT, Zakat, and e-invoicing regulation by retrieving relevant text from real regulatory documents and passing it to an LLM (retrieval-augmented generation, or RAG). Before the chatbot can answer anything correctly, the source documents have to be processed once, offline, into a searchable form. That processing step is the **offline ingestion pipeline**.

1.1 Why two stores: Qdrant + SQLite

The pipeline writes to two different databases, because the two kinds of content in these documents need to be searched in fundamentally different ways:

Store	What it holds	Why this store
Qdrant (vector database)	Prose paragraphs (chunked by article) and short *pointer* descriptions for tables	Semantic search — "find text similar in *meaning* to this question," not exact keyword matching. Each chunk is embedded into a 1024-dimension vector using the BGE-M3 model, and Qdrant finds the closest vectors to a query at search time.
SQLite (structured file DB)	The actual row-by-row data of every table in the source documents (penalty matrices, data dictionaries, etc.)	Tables don't embed well as prose — you don't want a vector search to fuzzily "summarize" a penalty amount. Instead, Qdrant stores a short pointer ("this table exists, here's what it's about"), and once retrieval decides that table is relevant, the exact row data is pulled from SQLite by that pointer's `table_ref` key.

1.2 The pipeline stages

Driven by `features/explainer/offline/ingest.py`, reading a `manifest.json` that lists every source file with its document type, language, version label, and whether it's the current version of that regulation:

1. **Extract** (`extract.py`) — PDFs go through **Docling** (with RapidOCR for any image-based content); XLSX/CSV go through pandas. Produces raw markdown text plus any tables found.
2. **Chunk** (`chunker.py`) — splits prose by regulation article number (e.g. "Article 14"). A fallback chunker catches documents with no numbered-article structure (overview/hub pages) so their content isn't silently dropped.
3. **Detect references** (`references.py`) — tags mentions of other articles inside each chunk's text, so the online retrieval step can later pull in cross-referenced context.
4. **Describe tables** (`table_descriptions.py`) — generates the short text pointer that represents each table in Qdrant.
5. **Embed** (`embed.py`) — every prose chunk and table pointer is embedded with **BGE-M3**. This must produce identical vectors to the already-deployed online Embedder for the same input, or search

quality degrades silently — so this code is a deliberate copy of the online embedding logic, not a reimplementation.

6. **Store** — chunks + vectors go to Qdrant (`qdrant_store.py`); table rows go to SQLite (`sqlite_store.py`).

> **Why deterministic IDs matter:** every point's ID is generated with `uuid5` from stable fields (document type, version, article number) rather than Python's built-in `hash()` , which is randomized per process. That makes re-running ingestion *idempotent* — a second run overwrites the same points instead of piling up duplicates.

1.3 Docker vs. running directly on the host

A Docker image already existed for this pipeline (`services/offline_data_ingestion/Dockerfile`), built on an NVIDIA CUDA base image with torch, Docling, and FlagEmbedding installed. The original plan was to run ingestion inside that container with GPU passthrough.

> **Why we switched to running it directly on the host instead:** this job is a *manual, monthly* batch process, run by one person on one machine — not an always-on service. A working conda environment (`torchEnv`) for this exact pipeline already existed and was previously validated (torch 2.13.0+cu126, transformers, FlagEmbedding, RTX 4060 Laptop GPU). Docker's GPU passthrough on Windows adds real friction (NVIDIA Container Toolkit, driver detection, image rebuild cycles) for a job that will only ever run on this one machine. Containerizing pays off when something needs to run on a different machine, a shared server, or be triggered automatically — none of which applies here.

1.4 The missing-compiler bug

> **What happened:** the first ingestion attempt (inside the Docker container) failed on every single document with `RuntimeError: Failed to find C compiler` . Docling's OCR step (RapidOCR) JIT-compiles a Triton GPU kernel the first time it runs — even for a GPU-only run — and that compile step needs a real C compiler in the container.
>
> **Root cause:** the Dockerfile already had a fix for this documented in a comment — installing `build-essential` — but that fix existed only as an **uncommitted local edit**. The actual running image had been built *before* that line was added, so `gcc` was genuinely missing from it (confirmed via `dpkg -l build-essential` inside the container, which showed it as not installed).
>
> **Decision:** rather than rebuild the image, this was the trigger to reconsider whether Docker was worth it at all for this job (see 1.3) — and switch to running it directly in the already-working `torchEnv` conda environment on the host, which has a real compiler and toolchain available natively.

1.5 Running the real ingestion

Before running for real, two gaps had to be closed:

- The only manifest with data actually present on disk pointed at files explicitly labeled "DUMMY TEST DATA" — not something you'd want feeding a production knowledge base.
- The real regulatory documents (16 files: VAT Implementing Regulations, GCC Unified VAT Agreement, e-invoicing/XML/security standards, Zakat regulations, SOCPA accountants' regulations, penalty tables, the e-invoice data dictionary, etc.) turned out to already be staged at `docs/rag_ingestion_data/`, along with a `manifest.json` correctly pointing at them and a `SOURCE_MANIFEST.md` documenting exactly where each file came from and what couldn't be gathered.

The command that was actually run, in `torchEnv`, against local Qdrant and the default local SQLite path:

```
QDRANT_URL=http://localhost:6333 conda run -n torchEnv --no-capture-output python -m
features.explainer.offline.ingest \
    --manifest docs/rag_ingestion_data/manifest.json \
    --embedding-model-path "C:\Users\kalim\models\bge-m3"
```

Piece	Purpose
`QDRANT_URL=http://localhost:6333`	Local Qdrant was already running via `docker-compose`, with its port published to `localhost` — no container networking needed since this now runs on the host, not inside Docker.
`conda run -n torchEnv`	Executes inside the validated GPU-ready conda environment without needing an interactive `conda activate`.
`--manifest`	Points at the real 16-document manifest, not the dummy one.
`--embedding-model-path`	Local filesystem path to the already-downloaded BGE-M3 checkpoint — avoids re-downloading a multi-GB model from Hugging Face.

1.6 Results

> **16 / 16 documents ingested successfully, zero failures.**
>
> - ~995 prose chunks and ~245 tables extracted across all documents
> - **1,240 points** written to the local Qdrant collection `know_vat_n_zatca`
> - An 872 KB SQLite database with real table data (penalty amounts, the e-invoice field dictionary, etc.)

Because the running `rag-online` container reads SQLite from a Docker-managed named volume (`know_vat_zatca_sqlite`) rather than a plain host path, the freshly generated SQLite file had to be copied into that volume with a disposable helper container:

```
docker run --rm -v know_vat_zatca_sqlite:/data -v "<host path to data dir>:/host" \
    alpine cp /host/know_vat_n_zatca.db /data/know_vat_n_zatca.db
```

Qdrant needed no such step — it's a running server, so the host-run pipeline wrote to it directly over the network. No service restart was required for either store: Qdrant serves live data, and SQLite reads open a fresh file connection per query rather than holding a stale cached handle.

PART 2

Deploying Mizan.ai on GCP

2.1 Why Kubernetes, why k3s

Locally, all ten backend services run via a single `docker-compose.yml`. That's fine for one developer's machine, but doesn't give you rolling restarts, self-healing, or a path to scaling past one box. The project's Kubernetes manifests (`k8s-gcp/`) convert every `docker-compose` service 1:1 into plain Deployments + Services — no Helm, nothing exotic — so the same mental model carries over.

> **Why k3s specifically:** k3s is a lightweight, single-binary Kubernetes distribution built for exactly this kind of single-node deployment — it bundles its own ingress controller (Traefik, see Part 3) and local-path storage provisioner, so there's no separate control plane to stand up. The whole cluster runs on one GCP Compute Engine VM (`k3s-vm` , `e2-highmem-2` , `us-central1-a`).

2.2 Cluster layout: namespace, config, secrets, images

Piece	What it's for
`namespace: mizan`	Every resource lives in one namespace, keeping it isolated from k3s's own system components.
`ConfigMap mizan-config`	Shared, *non-secret* values every service needs (DB host/port/name, LangChain project name) — one source of truth instead of repeating the same literal string in ten places.
`Secret mizan-secrets`	Everything sensitive (JWT secret, DB password, Mailgun/LangChain API keys, internal Modal endpoint URLs). Created once directly from the local `.env` file with `kubectl create secret generic --from-env-file` — the value never gets written into a YAML file or committed to the repo.
Artifact Registry	Every service's Docker image is built locally and pushed to `us-central1-docker.pkg.dev/mizan-ai-kalimi03/mizan-images/<service>:latest` , then pulled by the cluster using an `imagePullSecrets: gcr-json-key` reference on each Deployment.
PersistentVolumeClaims	`postgres-data` , `qdrant-data` , and `rag-online-sqlite` give the three stateful components durable storage that survives a pod restart.

Applying everything is one command, run via SSH on the VM once the manifests are staged there:

```
sudo kubectl apply -k /tmp/k8s-gcp/k8s-gcp
```

2.3 Getting the local knowledge base onto GCP

Running the ingestion pipeline (Part 1) only populated the *local* Qdrant and SQLite — the ones backing the developer machine's own `docker-compose` stack. The GCP deployment has its own, completely separate Qdrant and SQLite (the `rag-online-sqlite` PVC in particular started genuinely empty — this was a known, documented gap in the k8s manifests' own README before this work closed it). Getting the real data across required a few deliberate steps.

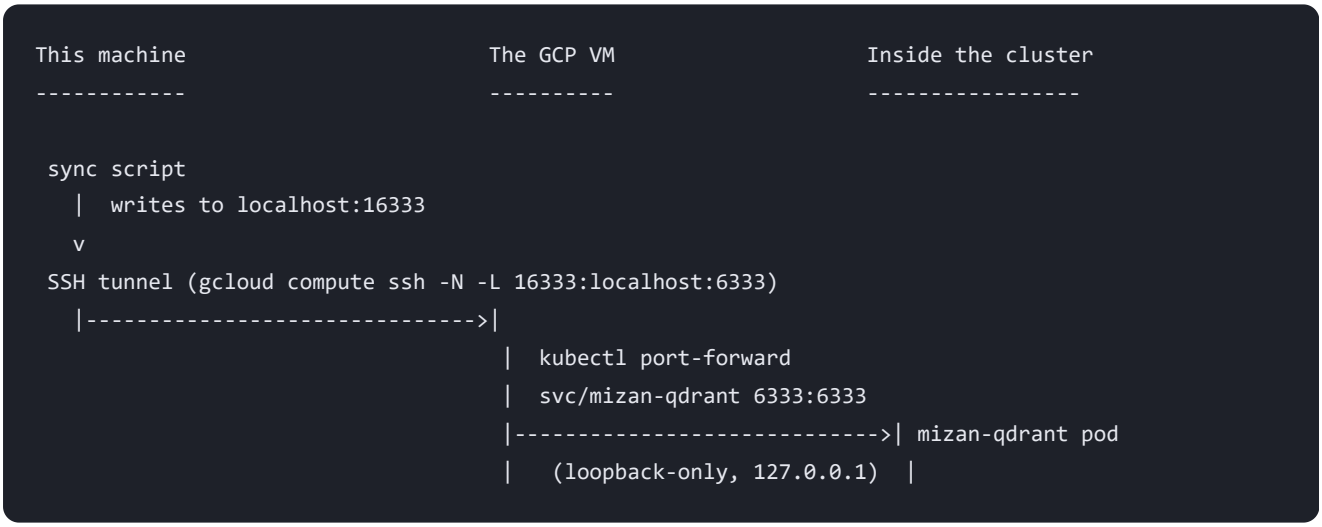
Reaching the cluster

> **Why `gcloud compute ssh` instead of the browser-based SSH-in-browser console:** the browser session only exists inside that browser tab — it can't be scripted or automated from a local terminal. `gcloud compute ssh` authenticates the same way but runs from the command line, so every step below could be executed and verified directly, rather than relying on manual copy-paste through a human in the loop.

```
gcloud compute ssh k3s-vm --zone us-central1-a --command "sudo kubectl get pods -n mizan"
```

Pushing Qdrant's data

Qdrant's Service on GCP is intentionally `ClusterIP` — not reachable from outside the cluster, on purpose (no reason to expose a raw vector database to the internet). Reaching it required a two-hop tunnel:



> **Why loopback-only, and why a different local port:** the port-forward on the VM was deliberately bound to `127.0.0.1`, not `0.0.0.0` — binding it to all interfaces would have exposed the GCP Qdrant instance on the VM's public IP, which the environment's own safety guardrails correctly refused to run. Local port `16333` (instead of the usual `6333`) was used purely to avoid colliding with the *local* Qdrant already listening on `6333` on the same machine.

With the tunnel open, a small one-off Python script (using `qdrant-client`) read every point — vector and payload — from the local collection via `scroll()`, created the destination collection if it didn't already have the right vector size/distance config, and `upsert()`'d everything through the tunnel in batches of 100:

```
src = QdrantClient(url="http://localhost:6333")    # local
dst = QdrantClient(url="http://localhost:16333")   # GCP, via tunnel
# ...scroll from src, upsert into dst, batch of 100 at a time...
```

> **Verified:** source and destination point counts matched exactly — 1,240 = 1,240.

> **Why a point-by-point sync instead of a Qdrant snapshot file:** at this data size (1,240 points), reading and re-writing through the client API is simpler than the alternative — export a binary snapshot, transfer the file, then restore it on the other side. No extra file format or transfer step to get wrong.

Pushing SQLite's data

SQLite isn't a server — it's just a file — so this was a straightforward two-hop file copy instead of a tunnel:

```
gcloud compute scp features/explainer/data/know_vat_n_zatca.db k3s-vm:/tmp/know_vat_n_zatca.db --
zone us-central1-a
gcloud compute ssh k3s-vm --zone us-central1-a --command \
  "sudo kubectl cp /tmp/know_vat_n_zatca.db mizan/<rag-online-pod>:/data/know_vat_n_zatca.db"
```

> **Verified:** the file landed at exactly 872,448 bytes — byte-for-byte identical to the local source. The temporary copy in the VM's `/tmp` was deleted immediately after. No pod restart was needed, for the same reason as the local sync: reads open a fresh file connection each time.

PART 3

Going Public: Cloudflare, DNS & TLS

3.1 Why a domain and a proxy in front

Up to this point, the only way to reach the deployment was the VM's raw IP address (`34.67.252.6`) with no encryption. Putting a real domain (`mizanai.org` , already registered and managed on Cloudflare) in front, proxied through Cloudflare, gives three things at once: a real HTTPS certificate visitors' browsers trust out of the box, the origin server's IP hidden from the public internet, and Cloudflare's edge network/DDoS protection sitting in front of the actual VM.

3.2 Path-based routing instead of subdomains

Every one of Mizan.ai's seven backend services could have been given its own subdomain (e.g. `chatbot.mizanai.org` , `calculator.mizanai.org` ...). That was deliberately avoided.

> **Why one domain with path-based routing was enough:** every backend service was already coded with its own, globally-unique `/api/...` path prefix, with zero overlap:

Path prefix	Service
`/api/auth` , `/api/feedback` , `/api/chat` , `/api/synthesis`	chatbot
`/api/compliance`	calculator
`/api/translate`	translator
`/api/convert`	converter
`/api/know-vat-zatca`	rag-online (explainer)
`/api/pdf-editor`	pdf-editor (editor)
`/api/comparator`	comparator
`/` (everything else)	frontend

Because these prefixes were already unique, the Ingress could route straight to the correct backend with **zero URL rewriting** — a single `mizanai.org` plus one TLS certificate covers every service, instead of needing a wildcard DNS record and wildcard certificate for per-service subdomains.

3.3 Traefik: the ingress controller already built into k3s

An "Ingress controller" is the piece of software that actually reads Kubernetes `Ingress` objects and turns them into real routing rules — Kubernetes' Ingress resource is just a spec, it does nothing on its own

without a controller watching it.

> **Why nothing extra was installed:** k3s ships with **Traefik** pre-installed as its default ingress controller, already running and already exposed on ports 80/443 via a `LoadBalancer` Service (confirmed with `kubectl get ingressclass`, which showed `traefik (default)`). Installing a second controller like `ingress-nginx` would have meant two controllers fighting over the same node ports for no benefit.

3.4 TLS strategy: Cloudflare Origin CA vs. Let's Encrypt

The standard way to get HTTPS on a Kubernetes ingress is **cert-manager** automatically requesting a certificate from **Let's Encrypt**. That was deliberately skipped here in favor of a simpler option made available specifically because Cloudflare already sits in front of the origin.

> **The reasoning:** Cloudflare already terminates HTTPS for every visitor at its own edge, using its free "Universal SSL" — the browser never talks to the origin server directly. That means the origin only needs a certificate *Cloudflare itself* trusts for the Cloudflare-to-origin hop — it doesn't need to be trusted by the public. Cloudflare gives this away for free as an **Origin CA certificate**, valid for 15 years, generated instantly with no domain-ownership challenge process at all. Using cert-manager here would have meant standing up an entire ACME HTTP-01/DNS-01 challenge flow to solve a problem Cloudflare had already solved for free.

How it was generated

1. Cloudflare dashboard → **mizanai.org** → **SSL/TLS** → **Origin Server** → **Create Certificate**
2. Defaults kept: Cloudflare generates the private key + CSR itself, key type RSA (2048), hostnames `mizanai.org` + `*.mizanai.org`, 15-year validity
3. Cloudflare returns two blocks: the **Origin Certificate** and the **Private Key**

3.5 Handling the private key safely

> **Why this mattered:** Cloudflare shows the private key exactly once — if you navigate away before saving it, it's gone for good and a new certificate has to be issued. It also needed to reach the cluster without ever being pasted into a chat transcript or printed to a terminal unnecessarily.

1. Certificate and key saved locally as two separate `.pem` text files
2. Verified as structurally valid PEM (checking only the `-----BEGIN..-----` / `-----END..-----` markers and line counts — never printing the actual key material)
3. Copied to the VM's `/tmp` with `gcloud compute scp`
4. Loaded into a Kubernetes TLS Secret:

```
sudo kubectl create secret tls mizanai-origin-tls -n mizan \
    --cert=/tmp/mizanai_origin.pem --key=/tmp/mizanai_origin_key.pem
```

5. The temporary files on the VM's `/tmp` deleted immediately afterward

3.6 DNS record and Cloudflare SSL mode

Setting	Value	Why
DNS record	Type `A`, name `@`, value `34.67.252.6`, **Proxied** (orange cloud)	"Proxied" routes all traffic through Cloudflare's network first — the origin's real IP is never exposed to visitors, and Cloudflare's edge features (caching, the free TLS cert, DDoS protection) only apply to proxied traffic. The alternative, "DNS only" (grey cloud), would point visitors straight at the origin IP with none of that.
SSL/TLS mode	**Full (strict)**	Cloudflare offers four modes: *Off* (no encryption at all), *Flexible* (encrypts browser↔Cloudflare only, plaintext Cloudflare↔origin — insecure), *Full* (encrypts both hops, but doesn't verify the origin cert is genuinely valid), and *Full (strict)* (encrypts both hops *and* requires a valid, trusted origin certificate). Full (strict) was the correct choice specifically because the Origin CA certificate from step 3.4 makes the origin's certificate valid and Cloudflare-trusted — without it, Full (strict) would reject the connection.

GCP's firewall already allowed inbound traffic on ports 80 and 443 to this VM (it already carried the `http-server` / `https-server` network tags from the original deployment), so no additional firewall changes were needed.

3.7 The Ingress resource itself

One Kubernetes `Ingress` object ties everything in this Part together — the TLS secret, the routing table from 3.2, and an annotation restricting it to the HTTPS entrypoint only:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: mizan-ingress
  annotations:
    traefik.ingress.kubernetes.io/router.entrypoints: websecure
spec:
  ingressClassName: traefik
  tls:
    - hosts: ["mizanai.org"]
      secretName: mizanai-origin-tls
  rules:
    - host: mizanai.org
      http:
        paths:
          - path: /api/know-vat-zatca
            pathType: Prefix
            backend: { service: { name: rag-online, port: { number: 8005 } } }
          # ...one entry per service, catch-all "/" routes to frontend
```

3.8 Deploying the frontend as a container

The frontend (plain HTML/CSS/JS, no framework) had never actually been deployed anywhere before this — locally it was just opened as static files, with `config.js` hardcoding each backend's URL to `localhost:<port>`. Getting it onto the cluster needed three new pieces:

1. **A Dockerfile** — a plain `nginx:alpine` image serving the static files.
2. **A production config override** (`config.prod.js`) — a second version of the config file where every backend base URL is set to an empty string instead of `localhost:PORT`. An empty base URL makes every fetch call same-origin and relative (e.g. `fetch("" + "/api/chat")` → `/api/chat`), which resolves correctly once the frontend and every API sit behind the same domain. The original `config.js` was left untouched so local development against `localhost` ports still works.
3. **A Deployment + Service** (`k8s-gcp/frontend.yaml`), added to the kustomization alongside the new `ingress.yaml` .

3.9 Two bugs hit along the way

> **Bug 1 — scp nesting a directory instead of updating it in place.**
> `gcloud compute scp --recurse k8s-gcp k3s-vm:/tmp/k8s-gcp` was meant to refresh the manifests already staged on the VM. Because `/tmp/k8s-gcp` already existed as a directory, scp copied the source folder *inside* it instead of overwriting its contents — the new `frontend.yaml` / `ingress.yaml` silently ended up at `/tmp/k8s-gcp/k8s-gcp/` , and the first `kubectl apply -k /tmp/k8s-gcp` ran against the stale, un-updated copy (visible from the apply output simply not mentioning frontend/ingress at all). Fixed by re-running `apply -k` against the correct, nested path.

> **Bug 2 — "Welcome to nginx!" instead of the app.**
> After deploying, `https://mizanai.org` showed nginx's own default placeholder page. Cause: none of the frontend's HTML files is literally named `index.html` — the base `nginx:alpine` image ships its own default `index.html`, and `COPY . /usr/share/nginx/html/` only adds/overwrites files with *matching* names, so that default page was never replaced. Fixed by explicitly copying `app.html` to `index.html` in the Dockerfile — chosen because its page title is simply "Mizan.ai" (a generic app shell) and `common.js` already redirects any unauthenticated visitor straight to `login.html`, so it's a safe landing page either way. After rebuilding and pushing the image, the fix also needed `kubectl rollout restart deployment/frontend` — reusing the same image tag (`:latest`) doesn't make a running pod re-pull it on its own.

3.10 Final end-to-end verification

```
Browser
  |  https://mizanai.org
  v
Cloudflare edge  (Universal SSL terminates here — public cert, DNS-proxied, hides origin IP)
  |  HTTPS, Full (strict) — validates the Origin CA cert below
  v
GCP VM 34.67.252.6 : 443  (firewall already open, http-server/https-server tags)
  |
  v
Traefik ingress  (k3s built-in, TLS via mizanai-origin-tls secret)
  |  path-based routing, no rewriting
  v
k8s Service  -->  Pod  (frontend nginx, or one of the 7 backend FastAPI services)
```

Check	Result
DNS resolution	`mizanai.org` resolves to Cloudflare's anycast IP ranges (confirms the record is proxied, not pointing directly at the origin)
Frontend	`https://mizanai.org/` → `200`, real app HTML
API routing	`https://mizanai.org/api/know-vat-zatca/chat` → `401` (the backend's own auth check responding — proof the request reached the real `rag-online` service through the full chain above, not a routing dead-end)